

BioVault: Biobank and Biospecimen Management System

Complete Database Design, Implementation, Testing, and Bonus Application

[REPLACE WITH STUDENT NAME]

Fundamentals of Databases — 2026

Contents

1	Submission information	3
2	Executive summary	3
3	1. Problem statement	3
4	2. Scope and stakeholders	4
4.1	2.1 In scope	4
4.2	2.2 Out of scope	4
4.3	2.3 Stakeholders	4
5	3. Assumptions and business rules	4
6	4. Conceptual design	5
6.1	4.1 Relationship and participation summary	5
7	5. Entities, attributes, and keys	6
8	6. Relational mapping and normalization	6
8.1	6.1 Relational mapping	6
8.2	6.2 First Normal Form	6
8.3	6.3 Second Normal Form	7
8.4	6.4 Third Normal Form	7
9	7. PostgreSQL implementation	7
9.1	7.1 Data types	7
9.2	7.2 Integrity constraints	7
9.3	7.3 Indexes	7
10	8. Test data	8
11	9. SQL operations	8
12	10. Views and advanced database objects	8
12.1	10.1 Views	8
12.2	10.2 Consent and inventory trigger	9
12.3	10.3 Audit trigger	9
13	11. Verification and evidence	9
14	12. Bonus connected user interface	10
15	13. Security, privacy, and ethics	11
16	14. Limitations and future work	11
17	15. Conclusion	11

18 References	12
19 Appendix A. Reproduction commands	12
20 Appendix B. Deliverable mapping	12

1 Submission information

Item	Value
Student name	[REPLACE WITH STUDENT NAME]
Student ID	[REPLACE WITH STUDENT ID]
Course	Fundamentals of Databases
Selected topic	Biobank and Biospecimen Management System
Project name	BioVault
Primary DBMS	PostgreSQL 16
Bonus application	Python/Tkinter desktop CRUD application
Data classification	Entirely synthetic; no real human data

2 Executive summary

BioVault is a relational database and connected desktop application for managing an anonymized research biobank. The system follows biospecimens through donor registration, versioned consent, collection, processing, aliquoting, physical storage, laboratory testing, and authorized research consumption. It addresses the central data-management problem of biobanking: a specimen must remain traceable and scientifically useful while use remains ethically authorized, inventory remains accurate, and direct donor identity stays outside the research database.

The PostgreSQL design contains 15 relations, including lookup and audit relations, and resolves the project/researcher many-to-many relationship through an associative table. It is normalized to Third Normal Form. Candidate keys, primary keys, foreign keys, unique constraints, checks, and targeted indexes protect both identity and performance. Three views provide operational inventory, project-usage, and consent summaries. A transactional trigger locks an aliquot before research use, validates active consent and project authorization, rejects inventory overdraw, and deducts quantity atomically. A separate generic trigger records critical updates and deletes in an audit history.

The submitted dataset is synthetic and includes at least ten meaningful records for each main table. Sixteen SQL demonstrations cover retrieval, joins, aggregation, subqueries, EXISTS, a recursive CTE, a window function, views, and reversible data modifications. Automated PostgreSQL tests prove important constraints and business rules. Eight Python tests prove the application data layer and complete donor CRUD lifecycle.

For the bonus, a Python/Tkinter application connects directly to PostgreSQL and provides live dashboard counts, searching, joined sample and test views, and validated create/update/delete operations. A zero-configuration SQLite mirror is included only so the user interface can be demonstrated immediately on a clean computer; PostgreSQL remains the academic source of truth.

3 1. Problem statement

Research biobanks collect biological material and associated data so approved studies can reuse high-quality specimens. A spreadsheet is inadequate because the same donor may have multiple consent documents and collection events; a sample may create multiple aliquots; a researcher may belong to multiple projects; and each physical use changes inventory. The system must also preserve history. Deleting the last assay row must not accidentally erase the specimen, project, or consent context.

BioVault therefore requires a database that can answer operational and governance questions such as:

- Which accepted aliquots remain available, in what quantity, and at which exact freezer location?
- Was general research use consent active on the date a specimen was consumed?
- Was the researcher assigned to the project on that date?
- Which samples and donors contributed to a project?
- Which test requests are pending or completed?
- Can an auditor reconstruct a critical change?

The project is designed for research inventory and governance, not patient care. It does not diagnose disease, bill patients, or store direct personal identifiers.

4 2. Scope and stakeholders

4.1 2.1 In scope

- pseudonymous donor registration and lifecycle status;
- versioned, typed consent documents with dates and restrictions;
- researchers, approved projects, and dated project membership;
- collection events with collector, site, protocol, and time;
- sample classification, processing, quality, and status;
- aliquot preparation, remaining quantity, physical position, and status;
- hierarchical facility, freezer, rack, and box storage;
- laboratory test requests and results;
- authorized research usage and automatic stock deduction;
- operational views, audit history, testing, and a connected interface.

4.2 2.2 Out of scope

- names, national IDs, addresses, phone numbers, or contact details;
- clinical diagnosis and treatment;
- billing, insurance, or appointment scheduling;
- instrument control and raw genomic files;
- regulatory approval workflow outside the stored ethics approval code;
- multi-site identity linkage.

4.3 2.3 Stakeholders

Stakeholder	Primary need
Biobank manager	inventory, consent visibility, auditability
Laboratory scientist	samples, aliquots, tests, storage locations
Principal investigator	project membership and usage summaries
Quality specialist	controlled values, rejected/quarantined material, evidence
Database administrator	reproducible deployment, constraints, indexes, backups
Instructor/auditor	correct design, explainable rules, repeatable tests

5 3. Assumptions and business rules

BioVault uses donor_code as a pseudonym. A separate authorized custodian would hold any identity-linking key; that key is not part of this project. All timestamps are stored using TIMESTAMPTZ. Quantities are compared only within the unit stored on an aliquot, and aggregate reports do not silently mix incompatible units.

The major rules are:

1. Every donor code is unique and matches BIO-D0000.
2. A donor may have several consent types and document versions.
3. Consent expiry cannot be earlier than grant.
4. Every collection event has exactly one donor and collector.
5. Every sample belongs to exactly one event and one sample type.
6. Sample and aliquot initial quantities must be positive.
7. Remaining aliquot quantity is between zero and its initial quantity.
8. One storage position can contain at most one aliquot.
9. Storage units form a self-referencing physical hierarchy.
10. Projects and researchers have a many-to-many relationship.
11. Each project has one lead and one unique ethics approval code.
12. Research use requires active RESEARCH_USE consent on the use date.
13. The user must be a project member on that date.
14. The project must be valid and authorized on that date.
15. Inventory use is concurrency-safe and cannot overdraw an aliquot.
16. Completed tests require a completion date on or after request.

17. Critical operational changes are audited.
18. Historical scientific records use restrictive deletion.

6 4. Conceptual design

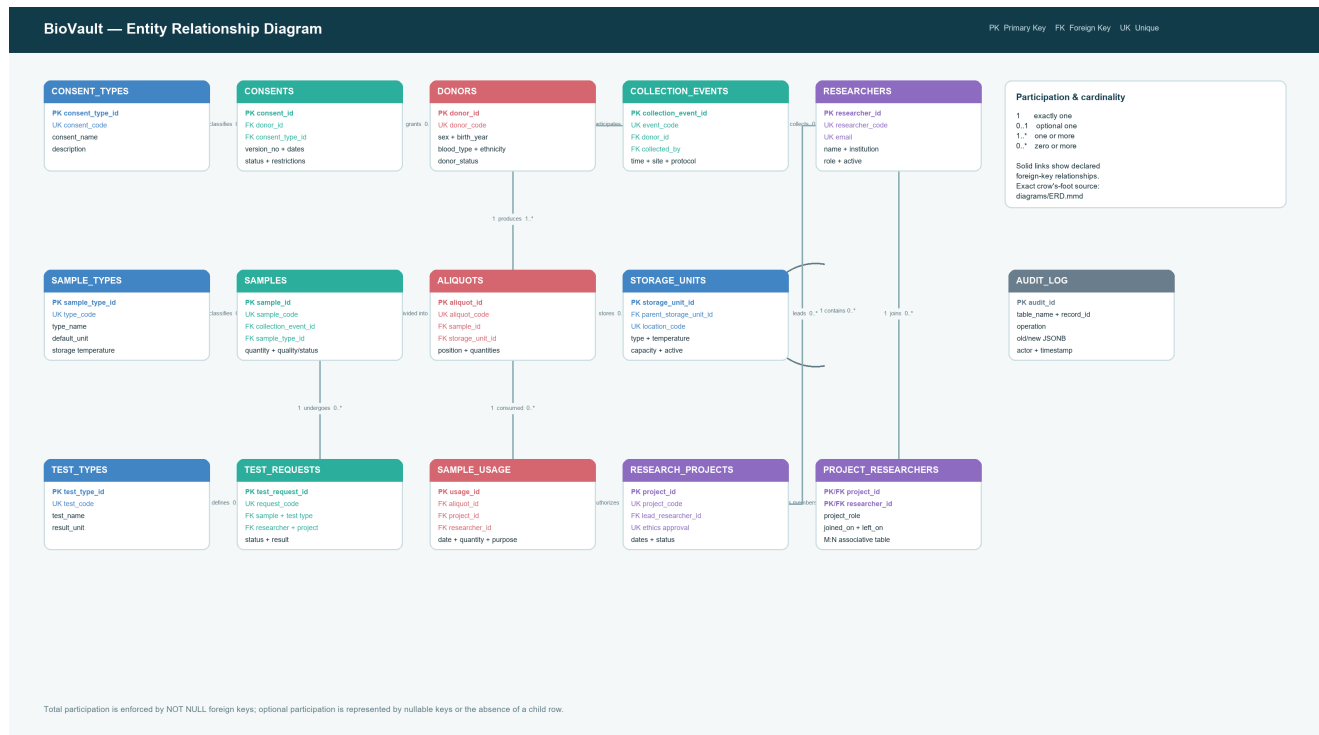


Figure 1: BioVault ER diagram. Exact crow's-foot source is included as diagrams/ERD.mmd.

The conceptual model separates people and authorization, biospecimen lineage, physical storage, testing, and consumption:

- **Donor/consent area:** donors, consent_types, and consents.
- **Scientific workforce:** researchers, research_projects, and project_researchers.
- **Specimen lineage:** collection_events, sample_types, samples, and aliquots.
- **Storage:** recursive storage_units.
- **Testing:** test_types and test_requests.
- **Controlled consumption:** sample_usage.
- **Technical history:** audit_log.

6.1 4.1 Relationship and participation summary

Relationship	Cardinality	Participation
Donor grants Consent	1 : 0..many	Consent total; donor partial
Consent Type classifies Consent	1 : 0..many	Consent total
Donor has Collection Event	1 : 0..many	Event total; donor partial
Event produces Sample	1 : 1..many by business expectation	Sample total
Sample Type classifies Sample	1 : 0..many	Sample total
Sample is divided into Aliquot	1 : 1..many by process	Aliquot total
Storage Unit stores Aliquot	1 : 0..many	Aliquot total
Storage Unit contains Unit	0..1 : 0..many	root parent is optional
Project has Researcher	many : many	resolved by bridge
Sample undergoes Test Request	1 : 0..many	Request total
Aliquot is consumed in Usage	1 : 0..many	Usage total
Project authorizes Usage	1 : 0..many	Usage total
Researcher records Usage	1 : 0..many	Usage total

NOT NULL foreign keys enforce total child participation. Optional participation is represented by a nullable key, as with an optional project on a test request, or by the absence of a child row.

7 5. Entities, attributes, and keys

Entity	Identifier	Candidate keys and main descriptive attributes
Donor	donor_id	donor_code; sex, birth year, blood type, ethnicity, status
Consent Type Consent	consent_type_id consent_id	consent_code; name, description donor/type/version; dates, status, restrictions
Researcher Research Project	researcher_id project_id	code, email; name, institution, role project code, ethics code; title, dates, status
Project Researcher Sample Type	project + researcher sample_type_id	role and membership dates type code; name, unit, storage range
Collection Event	collection_event_id	event code; donor, collector, time, site, protocol
Sample	sample_id	sample code; event, type, quantity, quality/status
Storage Unit	storage_unit_id	location code; parent, type, temperature, capacity
Aliquot	aliquot_id	aliquot code and storage position; quantities/status
Test Type Test Request	test_type_id test_request_id	test code; name and result unit request code; sample, test, requester, dates/result
Sample Usage	usage_id	aliquot, project, researcher, date, quantity, purpose
Audit Log	audit_id	logical table/record pair; before/after JSON and actor

Identity primary keys make joins compact and stable. Human-readable codes are retained as unique candidate keys. Foreign-key columns are explicitly indexed when they support expected joins.

8 6. Relational mapping and normalization

8.1 6.1 Relational mapping

Strong entities became tables with identity primary keys. One-to-many relationships place a foreign key on the many side: for example, `collection_events.donor_id`, `samples.collection_event_id`, and `aliquots.sample_id`.

The project/researcher many-to-many relationship became `project_researchers(project_id, researcher_id, project_role, joined_on, left_on)`. Its composite primary key prevents duplicate membership while its attributes describe the relationship itself.

The storage hierarchy became a recursive relationship through `storage_units.parent_storage_unit_id`. This supports arbitrary physical depth without separate tables for every freezer/rack/box level. Query 9 reconstructs complete paths with a recursive CTE.

8.2 6.2 First Normal Form

All attributes are atomic. Repeating consents, project members, aliquots, tests, and usages were separated into their own rows and relations. Each table has a declared key.

8.3 6.3 Second Normal Form

Most tables have a single-column primary key. In the composite bridge, role and membership dates depend on the complete (project_id, researcher_id) key. Researcher facts are not copied into that table, and project facts are not copied into it.

8.4 6.4 Third Normal Form

Lookup facts are separated by determinant:

- consent name depends on consent_type_id, not on a consent record;
- sample name, unit, and storage range depend on sample_type_id;
- test name and result unit depend on test_type_id;
- donor information is reached from a sample through its collection event;
- storage descriptions depend on storage_unit_id;
- project and researcher descriptions are not copied into sample_usage.

Therefore non-key attributes depend on the key, the whole key, and nothing but the key. The audit_log is an intentional technical exception: JSON snapshots are stored specifically to preserve the exact historical row state, not to act as master data.

9 7. PostgreSQL implementation

The implementation is divided into deterministic scripts:

1. create_tables.sql recreates the biobank schema, tables, constraints, and indexes.
2. triggers_procedures.sql installs timestamps, usage validation, inventory deduction, the public function, and audit triggers.
3. load_data.sql loads synthetic records and calls the real usage function.
4. views.sql creates the three read models.
5. setup.sql executes all files in the correct order.

9.1 7.1 Data types

- BIGINT ... AS IDENTITY supports compact generated primary keys.
- NUMERIC(14,3) protects quantities from binary floating-point error and permits cell counts above ten million.
- DATE models consent, project, and administrative validity.
- TIMESTAMPTZ models real-world event time.
- JSONB stores audit snapshots with PostgreSQL-native structure.
- BOOLEAN stores active flags.

9.2 7.2 Integrity constraints

The DDL names each important constraint. Examples include:

```
CONSTRAINT ck_aliquots_quantities CHECK (  
    initial_quantity > 0  
    AND current_quantity >= 0  
    AND current_quantity <= initial_quantity  
)
```

```
CONSTRAINT uq_aliquots_position  
    UNIQUE (storage_unit_id, position_code)
```

Foreign keys use restrictive deletion for donor/specimen history. Cascading deletion is used only for project membership, which cannot exist without its project.

9.3 7.3 Indexes

Thirteen targeted indexes support consent validity checks, donor event history, sample status, storage traversal, sample-to-aliquot lookup, test workflow, project usage, membership lookup, and audit history. A partial index covers available aliquots with stock:

```
CREATE INDEX idx_aliquots_availability
ON aliquots (aliquot_status, current_quantity)
WHERE aliquot_status = 'AVAILABLE';
```

10 8. Test data

The data is synthetic, internally coherent, and deliberately includes boundary states:

Main table	Rows
Donors	12
Researchers	10
Research projects	10
Project memberships	20
Collection events	12
Samples	20
Storage units	14
Aliquots	20
Test requests	15
Sample usage events	12

Lookup tables have four consent types, eight sample types, and six test types, which is logically sufficient. Donor 11 has expired research consent and donor 12 has withdrawn consent; their samples remain for traceability but research use is blocked. Test requests cover requested, in-progress, completed, and cancelled states. Storage spans -20 °C, -80 °C, and liquid-nitrogen locations.

11 9. SQL operations

`queries.sql` contains sixteen repeatable demonstrations:

- accepted/available sample retrieval;
- six-table lineage and location join;
- many-to-many project membership join;
- type-level sample and aliquot aggregation;
- GROUP BY ... HAVING for projects using multiple donors;
- correlated subquery against unit-specific average stock;
- two EXISTS predicates for material plus consent;
- DENSE_RANK window reporting;
- recursive storage-path CTE;
- CASE operational stock classification;
- direct use of the inventory view;
- filtered average test turnaround;
- reversible insert, update, and delete;
- reversible business-function execution.

All modification demonstrations run inside BEGIN ... ROLLBACK, so the script can be repeated during defense without changing the submitted dataset.

12 10. Views and advanced database objects

12.1 10.1 Views

`v_available_inventory` joins accepted samples, remaining aliquots, sample type, donor code, collection event, and physical location. It is the natural read model for inventory search.

`v_project_usage_summary` reports usage events, distinct samples, distinct donors, and total quantity for every project, including projects with no usage.

`v_donor_consent_status` exposes a current Boolean research-consent flag and latest expiry without exposing direct identity.

12.2 10.2 Consent and inventory trigger

The intended consumption entry point is:

```
SELECT record_sample_usage(  
    1, 1, 1, DATE '2026-06-15', 0.050,  
    'Defense demonstration'  
);
```

Before the row is inserted, `fn_validate_and_apply_sample_usage()`:

1. selects and locks the aliquot FOR UPDATE;
2. rejects missing, exhausted, or destroyed stock;
3. rejects a requested quantity above the remaining quantity;
4. validates project dates and status;
5. validates dated project membership;
6. validates active general research consent on the use date;
7. deducts quantity and marks the aliquot exhausted when it reaches zero.

The row lock prevents two concurrent sessions from both reading the same old quantity and overdrawing it. All work occurs in one database transaction, so a later failure rolls the deduction back.

12.3 10.3 Audit trigger

A generic trigger converts old and new rows to JSONB and records update/delete history for donors, samples, and aliquots. The trigger determines the appropriate primary-key field based on its table, which avoids duplicating three nearly identical functions.

13 11. Verification and evidence

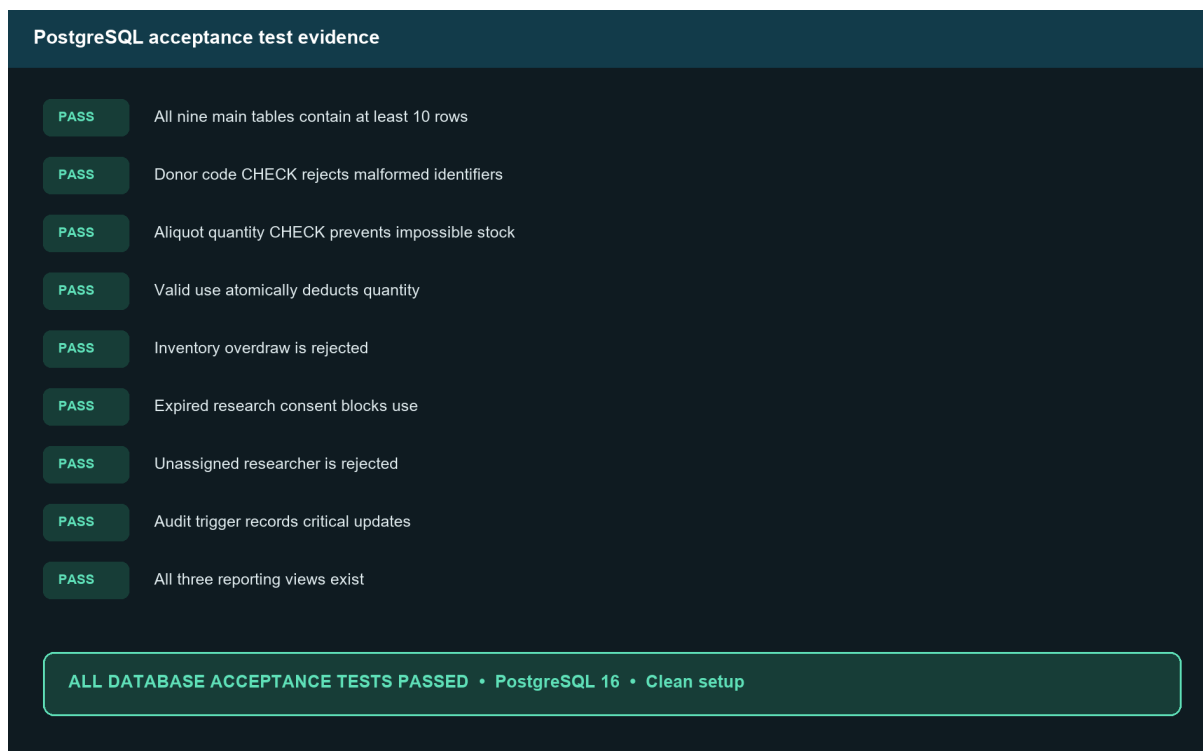


Figure 2: PostgreSQL acceptance-test result.

The database was built from an empty schema in PostgreSQL 16. Every statement in `setup.sql`, `test_constraints.sql`, and `queries.sql` completed successfully. The acceptance suite uses database-side assertions and stops immediately on an unexpected result.

Test	Expected behavior	Verified result
Main-table counts	nine main tables have ≥ 10 rows	Pass
Donor code domain	malformed code rejected	Pass
Quantity domain	current > initial rejected	Pass
Valid usage	exact stock deduction	Pass
Overdraw	rejected without stock change	Pass
Expired consent	research use rejected	Pass
Unauthorized member	research use rejected	Pass
Audit	update creates history row	Pass
Views	all three views exist	Pass

The Python test suite reports 8 passed. It verifies dashboard counts, joined search, create/read/update/delete round-trip, four validation failures, and foreign-key protection of referenced donors.

14 12. Bonus connected user interface

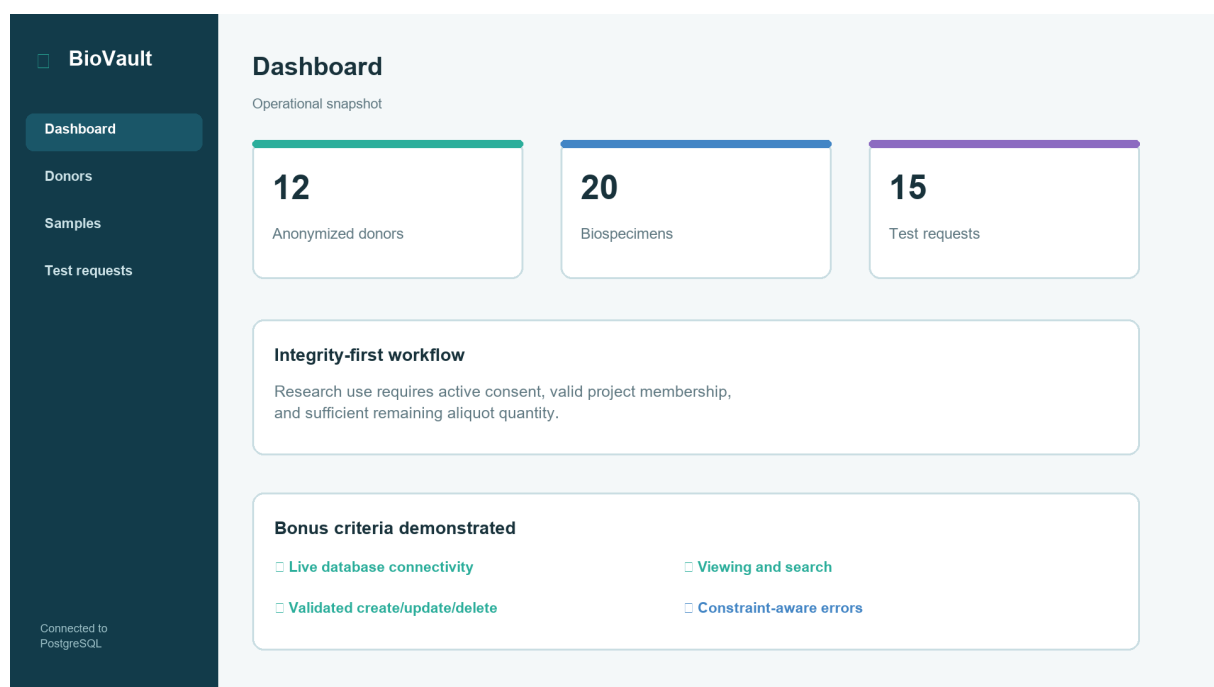


Figure 3: BioVault desktop dashboard preview.

The desktop application uses a three-layer structure:

- `app.py` contains the Tkinter presentation and user events.
- `repository.py` contains validation and parameterized data operations.
- `database.py` provides transaction-managed PostgreSQL/SQLite connections.

The UI satisfies all five bonus criteria:

Bonus criterion	Evidence
Database connectivity	live dashboard counts and PostgreSQL URL support
Viewing/search	donors, joined samples, and joined test requests
CRUD	validated donor create, read, update, and delete
Usability/validation	controlled choices, format/date/year checks, clear errors
Integration quality	same repository queries run against submitted PostgreSQL

Deletion errors are not hidden. Attempting to delete a donor referenced by a collection event produces a clear database message, demonstrating that the UI respects rather than bypasses integrity constraints.

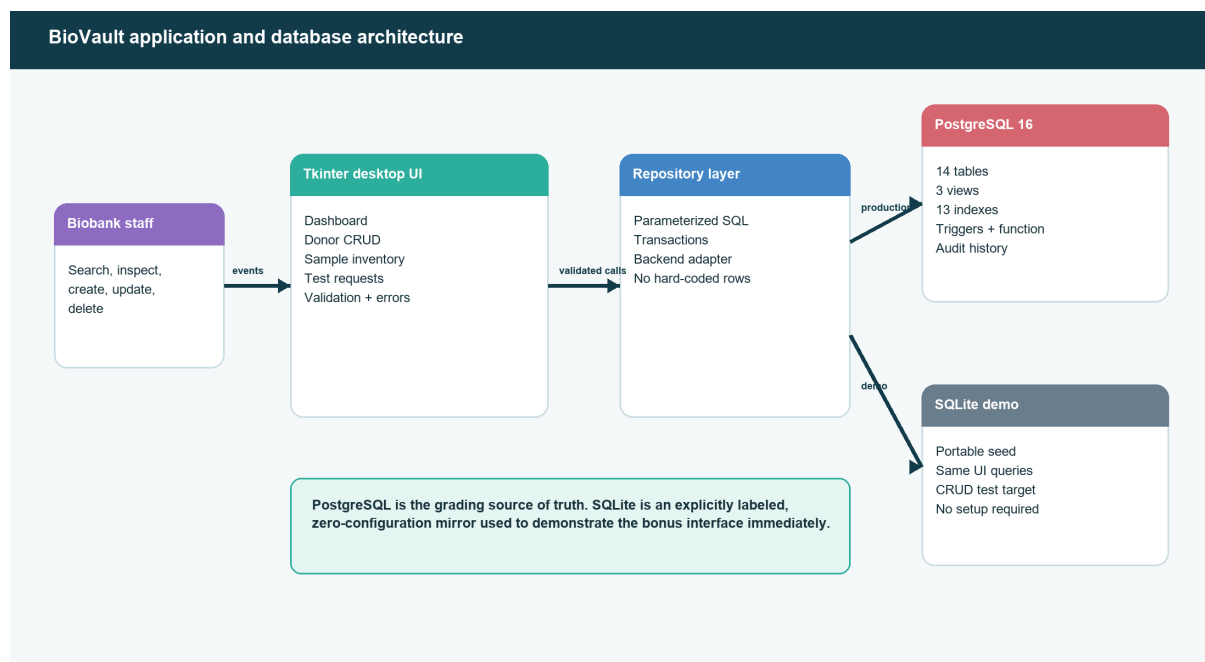


Figure 4: Application architecture.

15 13. Security, privacy, and ethics

The design follows data minimization: only research-relevant pseudonymous attributes are stored. Direct identity and re-identification keys are explicitly outside scope. PostgreSQL constraints implement defense in depth so a client cannot bypass the most important rules. Parameterized application queries avoid string-built SQL. Audit records support accountability.

A production system would also require TLS, role-based access, secrets management, encrypted backups, retention rules, disaster recovery, validated SOPs, jurisdiction-specific review, and possibly row-level security. Those are important but beyond this course project's local deployment.

16 14. Limitations and future work

- Units are stored per sample/aliquot; a production system may add a unit conversion catalog.
- The storage hierarchy does not yet validate that every child type is legal for its parent.
- Test results use generic numeric/text fields instead of assay-specific result schemas.
- The desktop application focuses CRUD on donors and read/search on other areas to keep the bonus interface understandable.
- The project does not implement authentication; it assumes a trusted local demonstration environment.

Future work could add role-based access, barcode scanning, temperature-sensor events, chain of custody, specimen reservation, test result attachments, and a web API.

17 15. Conclusion

BioVault completes the full database development lifecycle. Requirements and business rules are mapped to a readable ER model, a correct 3NF relational schema, executable PostgreSQL, synthetic data, varied SQL operations, transactional business logic, repeatable tests, documentation, and a connected CRUD application.

The strongest design decision is to enforce consent, researcher authorization, and inventory deduction inside PostgreSQL rather than only in the interface. That keeps the rule consistent for every future client and makes the behavior demonstrable during defense. The repository can be rebuilt from a clean database and verified with one script.

18 References

1. National Cancer Institute. *NCI Best Practices for Biospecimen Resources, Fourth Edition*. 2026. <https://dctd.cancer.gov/data-tools-biospecimens/biospecimens-biobanks/resources/best-practices/biospecimen-resources/appendices/2026-4th-edition-best-practices.pdf>
2. NCI Biorepositories and Biospecimen Research Branch. *Best Practices*. <https://biospecimens.cancer.gov/bestpractices/overview.asp>
3. ISO. *ISO 20387:2018 Biotechnology — Biobanking — General requirements for biobanking*. <https://www.iso.org/standard/67888.html>
4. PostgreSQL Global Development Group. *PostgreSQL Documentation: Data Definition*. <https://www.postgresql.org/docs/current/ddl.html>
5. PostgreSQL Global Development Group. *PostgreSQL Documentation: Triggers*. <https://www.postgresql.org/docs/current/triggers.html>
6. Python Software Foundation. *tkinter — Python interface to Tcl/Tk*. <https://docs.python.org/3/library/tkinter.html>

19 Appendix A. Reproduction commands

```
docker compose up -d database
docker compose exec -T database psql -U biobank -d biobank -f /project/sql/setup.sql
docker compose exec -T database psql -U biobank -d biobank -f /project/sql/test_constraints.sql
docker compose exec -T database psql -U biobank -d biobank -f /project/sql/queries.sql
```

PostgreSQL-connected bonus application:

```
$env:BIOBANK_DATABASE_URL = "postgresql://biobank:biobank_dev@localhost:55432/biobank"
python -m src.app
```

20 Appendix B. Deliverable mapping

Required deliverable	Repository location
Project overview and instructions	README.md
Complete report	report.docx, report.pdf
Presentation	presentation.pptx
Tables and constraints	sql/create_tables.sql
Test data	sql/load_data.sql
SQL operations	sql/queries.sql
Views	sql/views.sql
Trigger/function	sql/triggers_procedures.sql
ER diagram	diagrams/ERD.png, diagrams/ERD.mmd
Bonus application	src/
Automated tests	tests/, sql/test_constraints.sql
Video and defense material	presentation/